Student Name: Manideep Chitumala
Student ID: 24180624

MULTILAYER NEURAL NETWORKS: ANALYSIS OF DEPTH, WIDTH AND ACTIVATION FUNCTIONS

1. Introduction and Motivation

Multilayer neural networks, commonly known as multilayer perceptron's (MLPs), are widely used in machine learning for modelling complex relationships between inputs and outputs. Unlike simpler models, MLPs can capture nonlinear patterns by combining multiple layers of neurons, making them effective for tasks such as classification and regression. As illustrated in Figure 1, an MLP processes input features through multiple layers to produce predictions, allowing it to learn decision boundaries that separate different classes.

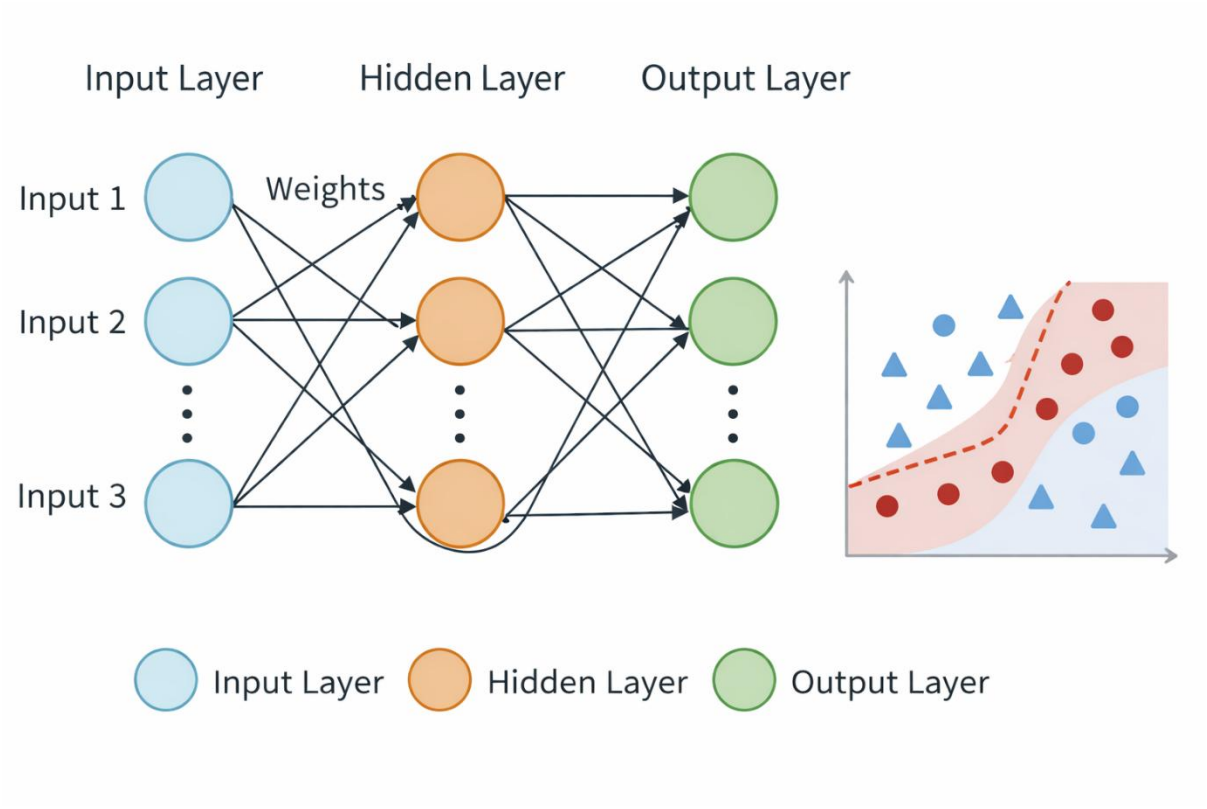

***Figure 1**: Structure of a multilayer neural network showing input, hidden, and output layers, along with a learned nonlinear decision boundary for classification.*

At a high level, an MLP works by passing input data through a sequence of weighted connections and nonlinear activation functions. During training, the network adjusts its weights to minimise prediction error, allowing it to learn patterns directly from data. However, designing an effective neural network is not always straightforward.

It might seem that increasing the number of layers or neurons would always improve performance, but in practice this can lead to problems such as overfitting or unstable training. This raises an important question: how do architectural choices influence model behaviour?

Although the Universal Approximation Theorem suggests that even a single hidden-layer network can approximate many functions, it does not guarantee efficient learning or good generalisation. In practice, factors such as depth, width, and activation functions play a critical role.

This tutorial investigates how these components affect neural network performance through both theoretical discussion and experimental analysis. The goal is to provide practical insights into how neural networks should be designed for effective use.

2. Neuron Model and Network Architecture

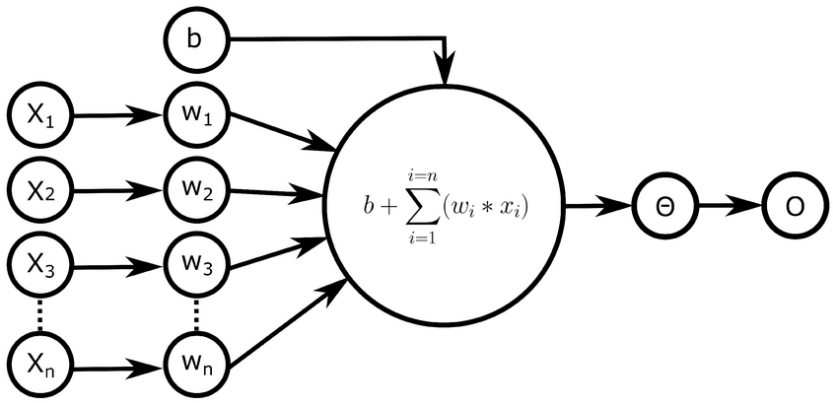

***Figure 2**: Structure of an artificial neuron and a fully connected multilayer perceptron, showing how inputs are transformed through weighted connections and activation functions across layers.*

A neural network is built from artificial neurons, which perform simple computations. Each neuron takes input values, multiplies them by weights, adds a bias term, and passes the result through an activation function. This process allows the network to transform input data into meaningful outputs.

Neurons are arranged into layers: an input layer, one or more hidden layers, and an output layer. The hidden layers are responsible for learning patterns, while the output layer produces predictions.

Two key architectural factors define an MLP:

- **Depth**: number of hidden layers
- **Width**: number of neurons per layer

Increasing depth allows the network to learn hierarchical features, where simple patterns are combined into more complex ones. Increasing width enables the network to capture more detailed variations within the data.

However, without activation functions, stacking layers would still result in a linear model. This highlights the importance of nonlinearity in neural networks.

3. Activation Functions

Activation functions are a key component of multilayer neural networks, as they introduce nonlinearity and allow the model to learn complex patterns. Without activation functions, even a network with multiple layers would behave like a linear model, limiting its ability to represent real-world data.

They also play an important role during training. Since gradients are propagated through the network, the choice of activation function directly affects how efficiently the model learns.

3.1 Sigmoid Function

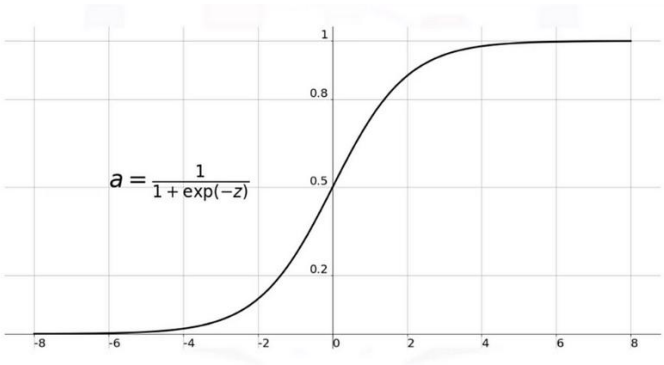

Figure 3.1: *Sigmoid activation function showing its S-shaped curve and saturation at extreme values, where gradients become very small and slow down learning.*

The sigmoid function maps input values to the range (0, 1), making it useful for probabilistic outputs such as binary classification. As shown in Figure 3.1, the function has an S-shaped curve that smoothly transforms inputs into output values between zero and one.

However, sigmoid suffers from **gradient saturation**. When input values become very large or very small, the curve flattens and gradients approach zero. This makes learning slow, particularly in deeper networks where gradients must pass through multiple layers. In practice, this limitation often makes sigmoid less suitable for hidden layers in modern neural networks.

3.2 Tanh Function

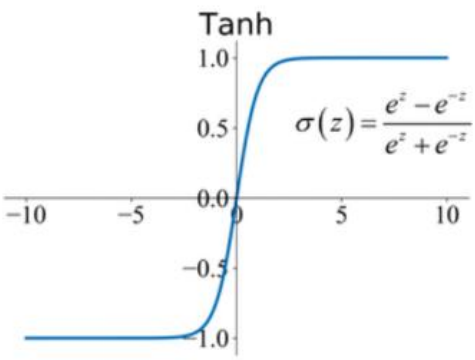

Figure 3.2: *Tanh activation function showing its zero-centred output range between -1 and 1, along with saturation at extreme values.*

The hyperbolic tangent (tanh) function outputs values in the range (-1, 1), making it zero-centred. As shown in Figure 3.2, this allows the function to produce both positive and negative outputs, which helps improve optimisation compared to sigmoid.

Because the outputs are centred around zero, gradients tend to be more balanced during training. This can lead to faster convergence and more stable learning in some cases. However, tanh still suffers from saturation when input values become very large or very small, causing gradients to approach zero.

As a result, although tanh generally performs better than sigmoid, it does not completely eliminate the vanishing gradient problem.

3.3 ReLU Function

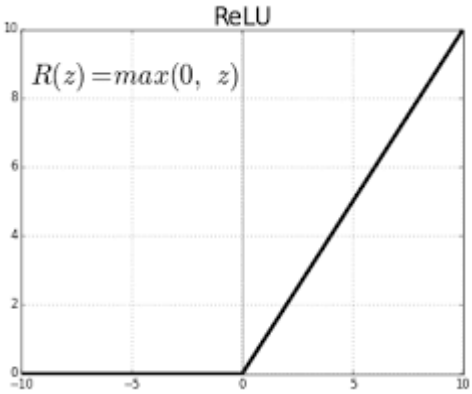

Figure 3.3: *ReLU activation function showing linear behaviour for positive inputs and zero output for negative inputs.*

ReLU (Rectified Linear Unit) outputs zero for negative inputs and increases linearly for positive inputs. It avoids saturation in the positive region, allowing gradients to remain strong and enabling faster training. However, ReLU can suffer from the **dying ReLU problem**, where neurons stop updating if they consistently output zero.

3.4 Comparative Insight

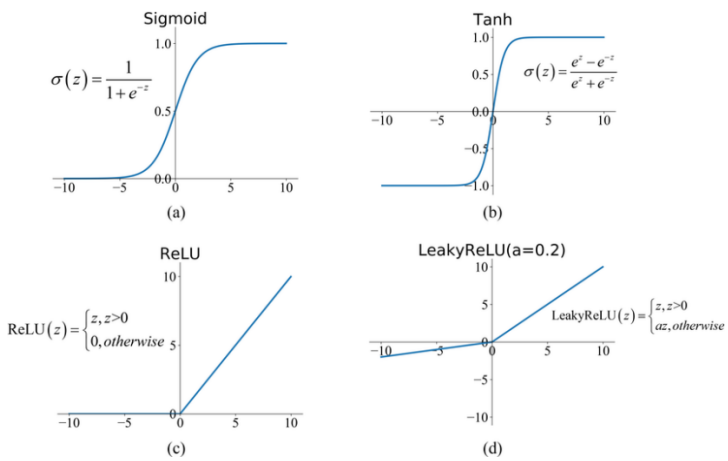

Figure 3.4: *Comparison of sigmoid, tanh, and ReLU functions, highlighting differences in gradient behaviour and output range.*

Comparing these activation functions highlights the trade-off between representation and optimisation. Sigmoid and tanh are smooth but prone to vanishing gradients, while ReLU provides a simpler structure that supports stable gradient flow.

This difference is also reflected in the experimental results, where ReLU and tanh outperform sigmoid. This suggests that activation functions which maintain stronger gradients tend to produce better learning performance.

4. Training with Loss Functions and Gradient Descent

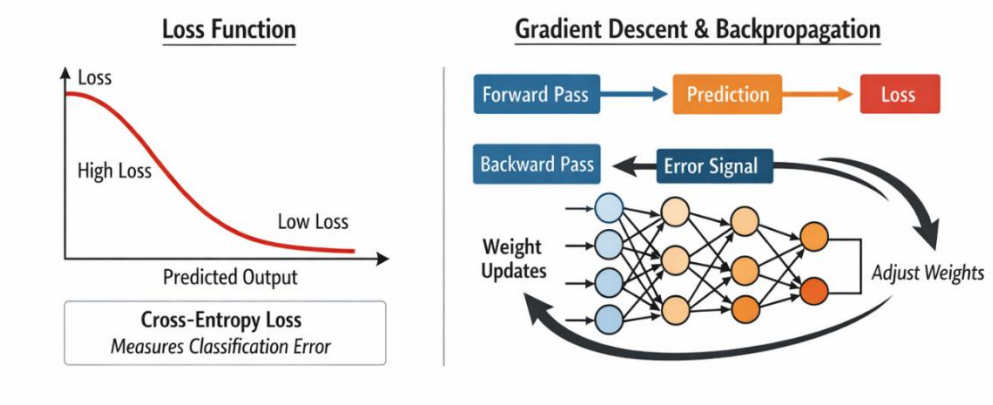

Figure 4: *Illustration of neural network training. The left side shows how the loss decreases as predictions improve, while the right side demonstrates the forward pass (prediction) and backward pass (error propagation) used to update model weights through gradient descent.*

Training a neural network involves minimising a loss function, which measures the difference between predicted and actual values. For classification tasks, cross-entropy loss is commonly used.

This optimisation is performed using gradient descent, which updates model parameters in the direction that reduces the loss. The training process consists of:

- **Forward pass**: compute predictions and loss
- **Backward pass (backpropagation)**: compute gradients and update weights

One challenge in training is maintaining stable gradients. In deep networks, gradients may become very small (vanishing gradients) or very large (exploding gradients), making training difficult.

The choice of activation function, learning rate, and other hyperparameters strongly affects training stability. Understanding these factors is essential for achieving good performance.

5. Capacity, Overfitting and Generalisation

Neural networks must balance their ability to fit training data with their ability to generalise to unseen data. This balance is determined by model capacity.

Capacity is influenced by depth and width. Increasing these increases the model's ability to learn complex patterns but 1`also introduces the risk of overfitting.

- **Underfitting**: model is too simple → poor performance
- **Overfitting**: model is too complex → memorises data
- **Good fit**: model captures underlying patterns without noise

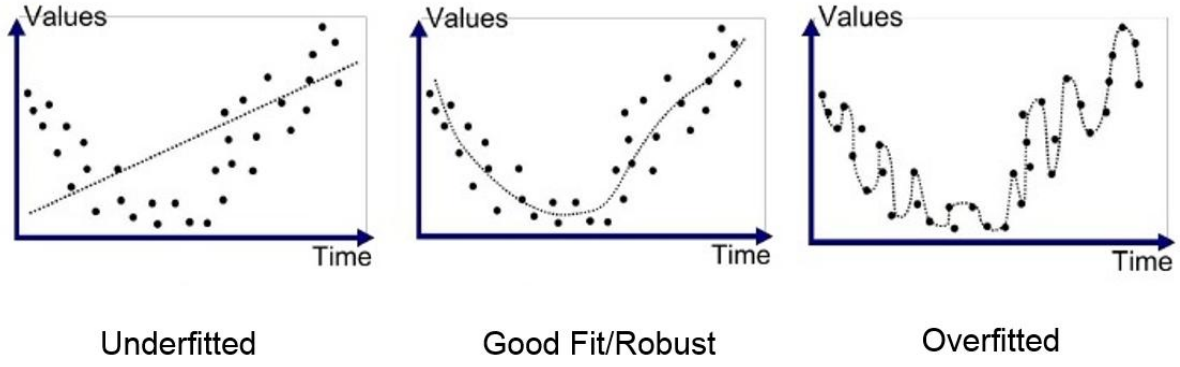

Figure 5: *Illustration of underfitting, optimal fitting, and overfitting. A simple model fails to capture the underlying pattern (underfitting), an appropriately complex model captures the trend effectively (good fit), and an overly complex model fits noise in the data rather than the true pattern (overfitting).*

As shown in Figure 5, increasing complexity improves performance up to a point, after which generalisation declines.
In this study, similar behaviour is observed. Increasing depth improves performance initially, but further increases provide limited benefit. Increasing width beyond a certain level also leads to diminishing returns. In practical applications, finding this balance is usually achieved through experimentation rather than relying only on theoretical expectations.
To reduce overfitting, techniques such as early stopping, L2 regularisation, and dropout can be used.
Overall, effective neural network design requires balancing model complexity with generalisation.

6. Experimental Study on a Dataset

To investigate the effects of architectural choices, experiments were conducted using the make_moons dataset. This dataset is non-linearly separable and suitable for visualising decision boundaries.

The dataset contains 1000 samples and was split into 80% training and 20% testing. Features were standardised, and models were implemented using MLP Classifier with max_iter=1000.

6.1 Effect of Network Depth

Three architectures were tested: (32,), (32,32), and (32,32,32).

- (32,): Train = 0.970, Test = 0.980
- (32,32): Train = 0.978, Test = 0.975
- (32,32,32): Train = 0.976, Test = 0.990

Increasing depth improved performance overall, with the three-layer model achieving the best test accuracy. However, improvements were not consistent at every step, indicating

diminishing returns. This suggests that while deeper networks can capture more complex patterns, the dataset itself does not require very deep architectures.

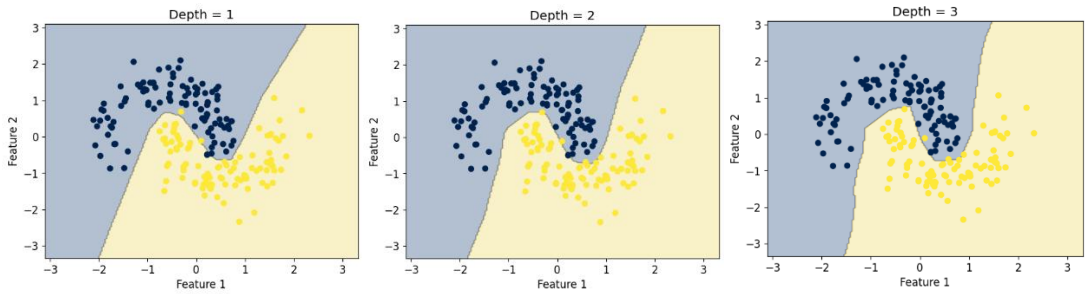

Figure 6.1: *Decision boundaries for networks with different depths. Increasing the number of hidden layers allows the model to represent more complex nonlinear patterns, although the improvement is not identical at every step.*

6.2 Effect of Network Width

Three widths were tested: (16,), (64,), (128,).

- (16,): Train = 0.970, Test = 0.980
- (64,): Train = 0.973, Test = 0.975
- (128,): Train = 0.973, Test = 0.975

Increasing width did not significantly improve performance. The smallest model performed slightly better on the test set, suggesting that the dataset does not require a large number of neurons. This indicates that once the model has enough capacity to represent the data, additional neurons may not provide meaningful benefits.

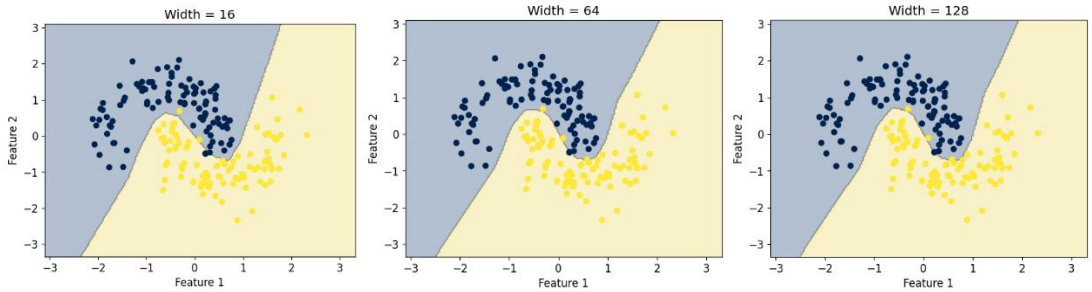

Figure 6.2: *Decision boundaries for networks with different widths. Increasing the number of neurons slightly changes the flexibility of the boundary, but beyond a certain point the additional capacity offers little improvement.*

6.3 Effect of Activation Functions

Three activation functions were compared: ReLU, tanh, and logistic.

- ReLU: Train = 0.976, Test = 0.990
- Tanh: Train = 0.971, Test = 0.990
- Logistic: Train = 0.870, Test = 0.850

ReLU and tanh performed well, while logistic showed significantly lower accuracy. This confirms that activation functions strongly influence training effectiveness. The results also suggest that stable gradient flow plays a key role in achieving good performance.

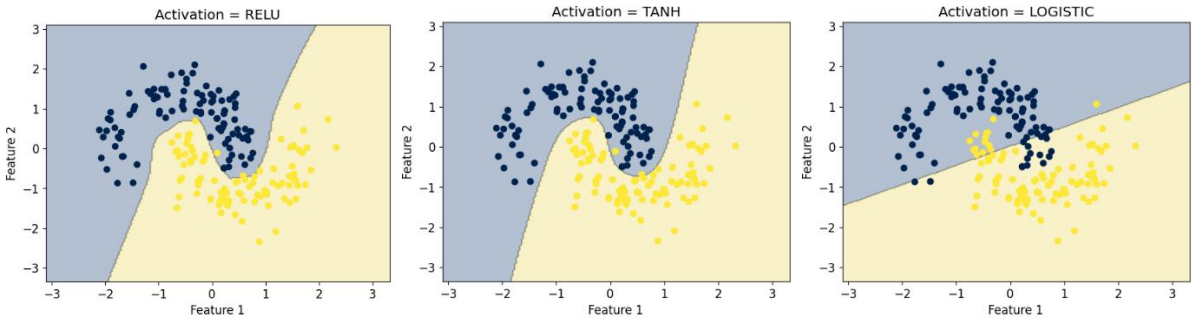

Figure 6.3*: Decision boundaries for different activation functions. ReLU and tanh capture the nonlinear pattern effectively, while logistic activation produces a weaker separation, reflecting its lower performance.*

6.4 Summary of Experimental Results

The experiments show that:

- Depth improves performance up to a point
- Width has limited impact beyond moderate size
- Activation function has the strongest effect

These results suggest that the dataset does not require a highly complex model, as simpler architectures already achieve strong performance. It also highlights that increasing complexity does not always lead to better generalisation, especially when the problem itself is relatively simple.

For clarity, the main quantitative results are summarised below.

Experiment	Configuration	Train Accuracy	Test Accuracy
Depth	(32,)	0.970	0.980
Depth	(32,32)	0.978	0.975
Depth	(32,32,32)	0.976	0.990
Width	(16,)	0.970	0.980
Width	(64,)	0.973	0.975
Width	(128,)	0.973	0.975
Activation	ReLU	0.976	0.990
Activation	Tanh	0.971	0.990
Activation	Logistic	0.870	0.850

6.1 Code and Repository

The complete implementation of the multilayer neural network experiments, including the dataset and Jupyter Notebook, is available at the following GitHub repository:

https://github.com/manichitumala/MULTILAYER-NEURAL-NETWORKS-ANALYSIS-OF-DEPTH-WIDTH-AND-ACTIVATION-FUNCTIONS.git

The repository contains all the necessary files required to reproduce the experiments presented in this report, including data generation, model implementation, and visualisation of results. A README file provides instructions for running the code.

7. Practical Guidelines for Using MLPs

In practice, designing an effective multilayer perceptron requires a balance between simplicity and performance. A useful starting point is to use a relatively simple architecture, such as one or two hidden layers with a moderate number of neurons, and then adjust the model based on observed results. If the model underfits, increasing depth or width can help improve its ability to capture more complex patterns.

However, if the model performs well on training data but poorly on test data, this indicates overfitting. In such cases, reducing model complexity or applying regularisation techniques such as dropout or early stopping can improve generalisation. It is also important to choose an appropriate activation function, as seen in the experiments where ReLU and tanh performed better than logistic.

Overall, neural network design should be guided by experimentation and validation rather than simply increasing model size.

8. Accessibility Considerations

This tutorial has been designed with accessibility in mind to ensure that it can be understood by a wide range of users.

All visualisations use clear, high-contrast colour schemes that are suitable for colour-blind readers. In particular, combinations such as blue and orange are used instead of red and green, which can be difficult to distinguish. All figures include descriptive captions, axis labels, and legends so that the information can be understood without relying solely on colour.

The structure of the report has been organised using clear headings and short paragraphs to improve readability. This makes it easier to follow the progression of ideas and supports users who rely on screen readers.

In addition, the accompanying code is written in a clear and readable manner, with comments explaining key steps. This allows users with different levels of programming experience to understand and reproduce the results.

By considering these aspects, the tutorial aims to be both informative and accessible, ensuring that the content can be effectively used by a broad audience.

9. Conclusion and Future Directions

This tutorial examined how key architectural components of multilayer neural networks—specifically depth, width, and activation functions—affect model behaviour and performance.

Through both theoretical discussion and experimental analysis, it became clear that increasing model complexity does not always lead to better results. Instead, effective performance depends on selecting an architecture that matches the problem.

The experimental results showed that increasing depth can improve performance by enabling the network to capture more complex patterns, although the benefit is not always consistent. In contrast, increasing width beyond a moderate level provided little improvement, suggesting that the dataset did not require a highly complex model. The comparison of activation functions highlighted their strong impact on training, with ReLU and tanh performing well, while logistic activation significantly reduced accuracy.

These findings emphasise that neural network design should focus on balancing capacity, generalisation, and training stability. In practice, this requires experimentation and careful evaluation rather than relying on assumptions about model size.

Overall, this study demonstrates that understanding both theoretical principles and practical behaviour is essential for building effective and reliable neural network models.

10. References

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Available at: http://neuralnetworksanddeeplearning.com/

GeeksforGeeks (n.d.). *Multilayer Perceptron (MLP) in Machine Learning*. Available at: https://www.geeksforgeeks.org/multilayer-perceptron-mlp/

IEEE. (2019). *Multilayer Perceptron Based Models for Prediction and Classification*. Available at: https://ieeexplore.ieee.org/document/8819775

ResearchGate. (n.d.). *Multilayer Perceptron and Neural Networks*. Available at: https://www.researchgate.net/publication/228340819_Multilayer_perceptron_and_neural_networks